



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №6
по курсу «Языки и методы программирования»
«Программа с графическим пользовательским интерфейсом»

Студент группы ИУ9-21Б Яннаев А. С.

Преподаватель Посевин Д. П.

Москва 2025

1 Цель работы

Приобретение навыков разработки программ с графическим пользовательским интерфейсом на основе библиотеки swing.

2 Задание

В течение лабораторной работы нужно разработать программу, рисующую на экране одно из изображений, перечисленных в таблицах 1 – 7. Программа должна иметь графический пользовательский интерфейс, через который пользователь может задавать параметры изображения. Изображение должно перерисовываться автоматически при изменении любого параметра.

3 Результаты

Исходный код программы представлен в 1, 2, 3, 4.

Листинг 1: Файл

```
1 import javax.swing.*;
2 import javax.swing.event.ChangeEvent;
3 import javax.swing.event.ChangeListener;
4
5 public class PictureForm {
6     private JSpinner sideASpinner;
7     private JSpinner sideBSpinner;
8     private JSpinner sideCSpinner;
9     private JPanel mainPanel;
10    private CanvasPanel canvasPanel;
11
12
13    public PictureForm() {
14        //
15
16        sideASpinner.setValue(100);
17        sideBSpinner.setValue(100);
18        sideCSpinner.setValue(100);
19        //
20
21        ChangeListener triangleChangeListener = new ChangeListener() {
```

```

21         public void stateChanged(ChangeEvent e) {
22             int a = (int) sideASpinner.getValue();
23             int b = (int) sideBSpinner.getValue();
24             int c = (int) sideCSpinner.getValue();
25
26             //
27
28             canvasPanel.setSides(a, b, c);
29
30             //
31             (
32             }
33         };
34
35         //
36
37         sideASpinner.addChangeListener(triangleChangeListener);
38         sideBSpinner.addChangeListener(triangleChangeListener);
39         sideCSpinner.addChangeListener(triangleChangeListener);
40
41         //
42
43         //canvasPanel.setSides(100, 100, 100);
44
45     }
46
47     public static void main(String[] args) {
48         JFrame frame = new JFrame("PictureForm");
49         frame.setContentPane(new PictureForm().mainPanel);
50         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51         frame.pack();
52         frame.setVisible(true);
53     }
54
55     private void createUIComponents() {
56         canvasPanel = new CanvasPanel();
57     }
58 }

```

Листинг 2: Файл

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.geom.Point2D;
4
5 public class CanvasPanel extends JPanel {

```

```

6      private int sideA = 100;
7      private int sideB = 100;
8      private int sideC = 100;
9
10     public void setSides(int a, int b, int c) {
11         this.sideA = a;
12         this.sideB = b;
13         this.sideC = c;
14         repaint();
15     }
16
17     protected void paintComponent(Graphics g) {
18         super.paintComponent(g);
19         Graphics2D g2d = (Graphics2D) g;
20
21         //
22
23         g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
24                               RenderingHints.VALUE_ANTIALIAS_ON);
25
26         //
27
28         if (!isValidTriangle(sideA, sideB, sideC)) {
29             g2d.setColor(Color.RED);
30             g2d.drawString("                :
31                               !", 20, 20);
32
33             return;
34         }
35
36         //
37         double angleA = calculateAngle(sideB, sideC, sideA);
38         double angleB = calculateAngle(sideA, sideC, sideB);
39         double angleC = calculateAngle(sideA, sideB, sideC);
40
41         //
42         int red = (int)(angleA * 255 / 180);
43         int green = (int)(angleB * 255 / 180);
44         int blue = (int)(angleC * 255 / 180);
45
46         //
47         red = Math.max(0, Math.min(255, red));
48         green = Math.max(0, Math.min(255, green));
49         blue = Math.max(0, Math.min(255, blue));
50
51         Color triangleColor = new Color(red, green, blue);

```

```

47
48      //
49      Point2D.Double[] vertices = calculateTriangleVertices();
50
51      //
52      g2d.setColor(triangleColor);
53      int[] xPoints = {(int)vertices[0].x, (int)vertices[1].x, (int)
vertices[2].x};
54      int[] yPoints = {(int)vertices[0].y, (int)vertices[1].y, (int)
vertices[2].y};
55      g2d.fillPolygon(xPoints, yPoints, 3);
56
57      //
58      g2d.setColor(Color.BLACK);
59      g2d.drawPolygon(xPoints, yPoints, 3);
60
61      //
62      drawAngleLabels(g2d, vertices, angleA, angleB, angleC);
63  }
64
65  private boolean isValidTriangle(int a, int b, int c) {
66      return a + b > c && a + c > b && b + c > a && a > 0 && b > 0 &&
c > 0;
67  }
68
69  private double calculateAngle(double a, double b, double c) {
70      return Math.toDegrees(Math.acos((a*a + b*b - c*c) / (2*a*b)));
71  }
72
73  private Point2D.Double[] calculateTriangleVertices() {
74      //
75
76      double maxSide = Math.max(sideA, Math.max(sideB, sideC));
77      double scale = (getWidth() * 0.8) / maxSide;
78
79      //
80      Point2D.Double a = new Point2D.Double(
81          getWidth() * 0.1,
82          getHeight() * 0.9
83      );
84
85      //
86      Point2D.Double b = new Point2D.Double(
87          a.x + sideA * scale,

```

```

87         a.y
88     );
89
90     //                                     (C)
91
92     double angleC = calculateAngle(sideA , sideB , sideC);
93     double cx = a.x + sideB * scale * Math.cos(Math.toRadians(angleC
94 ));
95     double cy = a.y - sideB * scale * Math.sin(Math.toRadians(angleC
96 ));
97     Point2D.Double c = new Point2D.Double(cx , cy);
98
99     //
100    double centerX = (a.x + b.x + c.x) / 3;
101    double centerY = (a.y + b.y + c.y) / 3;
102    double offsetX = getWidth()/2 - centerX;
103    double offsetY = getHeight()/2 - centerY;
104
105    a.x += offsetX;
106    a.y += offsetY;
107    b.x += offsetX;
108    b.y += offsetY;
109    c.x += offsetX;
110    c.y += offsetY;
111
112    return new Point2D.Double[]{ a , b , c };
113 }
114
115 private void drawAngleLabels(Graphics2D g2d , Point2D.Double[]
116 vertices ,
117                                     double angleA , double angleB , double
118 angleC) {
119     g2d.setColor(Color.BLACK);
120     g2d.setFont(new Font("Arial" , Font.BOLD, 12));
121
122     //                                     A (
123                                     )
124     drawLabelNearVertex(g2d , vertices[0] , vertices[1] , vertices[2] ,
125         String.format("A: %.1 f " , angleA));
126
127     //                                     B (
128                                     )
129     drawLabelNearVertex(g2d , vertices[1] , vertices[0] , vertices[2] ,
130         String.format("B: %.1 f " , angleB));
131
132     //                                     C (
133                                     )

```

```

126         drawLabelNearVertex(g2d, vertices[2], vertices[0], vertices[1],
127             String.format("C: %.1 f ", angleC));
128     }
129
130     private void drawLabelNearVertex(Graphics2D g2d, Point2D.Double
vertex,
131
132         Point2D.Double neighbor1, Point2D.
Double neighbor2,
133
134         String text) {
135         //
136
137         double dx1 = neighbor1.x - vertex.x;
138         double dy1 = neighbor1.y - vertex.y;
139         double dx2 = neighbor2.x - vertex.x;
140         double dy2 = neighbor2.y - vertex.y;
141
142         //
143         double len1 = Math.sqrt(dx1*dx1 + dy1*dy1);
144         double len2 = Math.sqrt(dx2*dx2 + dy2*dy2);
145         dx1 /= len1; dy1 /= len1;
146         dx2 /= len2; dy2 /= len2;
147
148         //
149         double bisectorX = (dx1 + dx2) / 2;
150         double bisectorY = (dy1 + dy2) / 2;
151
152         //
153         int labelX = (int)(vertex.x + bisectorX * 20);
154         int labelY = (int)(vertex.y + bisectorY * 20);
155
156         //
157         FontMetrics fm = g2d.getFontMetrics();
158         int textWidth = fm.stringWidth(text);
159         g2d.drawString(text, labelX - textWidth/2, labelY + fm.getAscent
() / 2);
160     }
161 }

```

Листинг 3: Файл

```

1 import javax.swing.*;
2 import javax.swing.event.ChangeEvent;
3 import javax.swing.event.ChangeListener;
4
5 public class PictureForm {
6     private JSpinner spinner1;
7     private JSpinner spinner2;

```

```

8      private JSpinner spinner3;
9      private JCheckBox checkBox1;
10     private JPanel mainPanel;
11     private CanvasPanel canvasPanel;
12
13     public PictureForm() {
14         //
15         spinner1.setValue(50); // radiusA
16         spinner2.setValue(40); // radiusB
17         spinner3.setValue(70); // distance
18         checkBox1.setSelected(false); // showIntersection
19
20         //
21         Runnable updateCanvas = () -> {
22             int radiusA = (int) spinner1.getValue();
23             int radiusB = (int) spinner2.getValue();
24             int distance = (int) spinner3.getValue();
25             boolean showIntersection = checkBox1.isSelected();
26
27             canvasPanel.setParameters(radiusA, radiusB, distance,
showIntersection);
28         };
29
30         //
31         spinner1.addChangeListener(new ChangeListener() {
32             public void stateChanged(ChangeEvent e) {
33                 updateCanvas.run();
34             }
35         });
36
37         spinner2.addChangeListener(new ChangeListener() {
38             public void stateChanged(ChangeEvent e) {
39                 updateCanvas.run();
40             }
41         });
42
43         spinner3.addChangeListener(new ChangeListener() {
44             public void stateChanged(ChangeEvent e) {
45                 updateCanvas.run();
46             }
47         });
48
49         checkBox1.addChangeListener(new ChangeListener() {
50             public void stateChanged(ChangeEvent e) {
51                 updateCanvas.run();
52             }

```



```

53         });
54
55         //
56         updateCanvas.run();
57     }
58
59     public static void main(String[] args) {
60         JFrame frame = new JFrame("PictureForm");
61         frame.setContentPane(new PictureForm().mainPanel);
62         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
63         frame.pack();
64         frame.setVisible(true);
65     }
66
67     private void createUIComponents() {
68         canvasPanel = new CanvasPanel();
69     }
70 }

```

Листинг 4: Файл

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.geom.Ellipse2D;
4 import java.awt.geom.Point2D;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 public class CanvasPanel extends JPanel {
9     private int radiusA = 50;
10    private int radiusB = 40;
11    private int distance = 70;
12    private boolean showIntersection = false;
13    private List<Point2D> intersectionPoints = new ArrayList<>();
14
15    public void setParameters(int radiusA, int radiusB, int distance,
16    boolean showIntersection) {
17        this.radiusA = radiusA;
18        this.radiusB = radiusB;
19        this.distance = distance;
20        this.showIntersection = showIntersection;
21        calculateIntersection();
22        repaint();
23    }
24
25    private void calculateIntersection() {
26        intersectionPoints.clear();

```

```

26
27     //
28     int centerX1 = 150;
29     int centerY1 = 150;
30     int centerX2 = centerX1 + distance;
31     int centerY2 = centerY1;
32
33     //
34     double a = radiusA;
35     double b = radiusB;
36     double d = distance;
37
38     if (d > a + b || d < Math.abs(a - b)) {
39         //
40         return;
41     }
42
43     double x = (d*d - b*b + a*a) / (2*d);
44     double y = Math.sqrt(a*a - x*x);
45
46     intersectionPoints.add(new Point2D.Double(centerX1 + x, centerY1
47 + y));
48     intersectionPoints.add(new Point2D.Double(centerX1 + x, centerY1
49 - y));
50 }
51
52 protected void paintComponent(Graphics g) {
53     super.paintComponent(g);
54     Graphics2D g2d = (Graphics2D) g;
55
56     //
57     int centerX1 = 150;
58     int centerY1 = 150;
59     int centerX2 = centerX1 + distance;
60     int centerY2 = centerY1;
61
62     //
63     g2d.setColor(Color.BLUE);
64     g2d.drawOval(centerX1 - radiusA, centerY1 - radiusA, 2*radiusA,
65 2*radiusA);
66
67     //
68     g2d.setColor(Color.GREEN);
69     g2d.drawOval(centerX2 - radiusB, centerY2 - radiusB, 2*radiusB,
70 2*radiusB);
71
72

```

```

68      //
69      if (showIntersection && !intersectionPoints.isEmpty()) {
70          g2d.setColor(Color.RED);
71          for (Point2D point : intersectionPoints) {
72              g2d.fill(new Ellipse2D.Double(point.getX() - 3, point.
73                  getY() - 3, 6, 6));
74          }
75      }
76  }

```

Результат запуска представлен на рисунке 1.

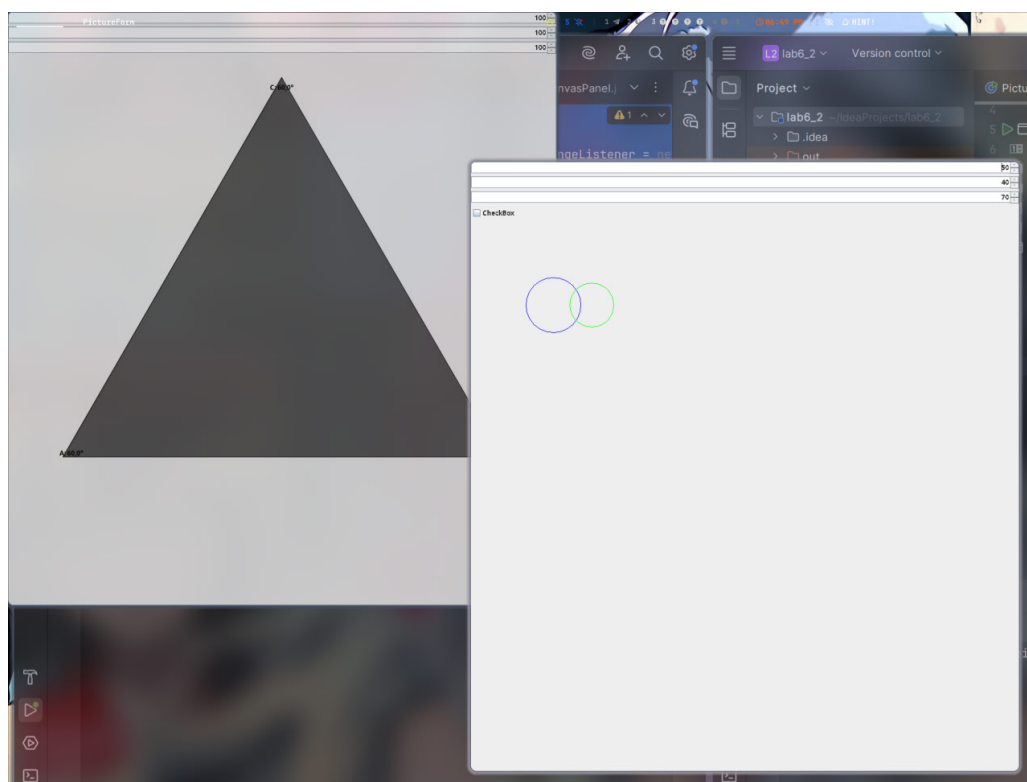


Рис. 1 — Результат